Project Report: PyTorch-to-TinyML Lightweight Compiler

Manvendra Pratap Singh, Alok Singh, Chandan Varma

Indian Institute of Technology Jodhpur
Jodhpur, India

***Abstract*—The deployment of deep learning models on highly constrained microcontrollers (MCUs) presents significant challenges. Modern deep neural networks, typically trained in high-level frameworks like PyTorch using 32-bit floating-point (FP32) arithmetic, consume megabytes of memory and require high computational power. Conversely, typical TinyML target devices (such as ARM Cortex-M microcontrollers) possess severely constrained resources, often featuring less than 256KB of SRAM, limited flash storage, and clock speeds under 100 MHz.**

I. Objective

The objective of this project is to bridge this gap by developing a custom, lightweight PyTorch-to-C compiler. This compiler automatically translates standard PyTorch models into optimized, dependency-free, INT8-quantized C code. The resulting code must strictly adhere to static memory constraints (no dynamic allocation) and execute efficiently within the strict hardware limitations of an MCU.

II. Solution Architecture & Methodology

To solve this problem, we designed a multi-stage compiler pipeline that systematically transforms a high-level computational graph into optimized, hardware-aware machine instructions (C code).

The pipeline is divided into seven sequential steps:

A. Step 1: Graph Extraction (Frontend)

We use `torch.fx` to symbolically trace the PyTorch model (`nn.Module`). This extracts a Directed Acyclic Graph (DAG) representing the mathematical operations and their data dependencies. We also run a dummy input through the traced graph to infer and record tensor shapes at every node.

```
(manvendra@Rahuls-MacBook-Pro tinyml_compiler % source venv/bin/activate
(venv) manvendra@Rahuls-MacBook-Pro tinyml_compiler % python3 main.py --model mnist

          TinyML Compiler v0.1.0
          PyTorch → Optimized INT8 C Code for MCU

Using model: SimpleCNN_MNIST (MNIST 28x28)

============================================================
STEP 1: Graph Extraction
============================================================
Graph: SimpleCNN_MNIST
Total nodes: 12

  x                | placeholder     | x                | inputs=[] | shape=(1, 1, 28, 28)
  conv1            | call_module     | Conv2d           | inputs=['x'] | shape=(1, 16, 28, 28)
  relu1            | call_module     | ReLU             | inputs=['conv1'] | shape=(1, 16, 28, 28)
  pool1            | call_module     | MaxPool2d        | inputs=['relu1'] | shape=(1, 16, 14, 14)
  conv2            | call_module     | Conv2d           | inputs=['pool1'] | shape=(1, 32, 14, 14)
  relu2            | call_module     | ReLU             | inputs=['conv2'] | shape=(1, 32, 14, 14)
  pool2            | call_module     | MaxPool2d        | inputs=['relu2'] | shape=(1, 32, 7, 7)
  flatten          | call_module     | Flatten          | inputs=['pool2'] | shape=(1, 1568)
  fc1              | call_module     | Linear           | inputs=['flatten'] | shape=(1, 128)
  relu3            | call_module     | ReLU             | inputs=['fc1'] | shape=(1, 128)
  fc2              | call_module     | Linear           | inputs=['relu3'] | shape=(1, 10)
  output           | output          | output           | inputs=['fc2'] | shape=(1, 10)

Time  Graph extraction: 19.5 ms
```

Fig. 1. Step 1: Graph Extraction Output

B. Step 2: Intermediate Representation (IR) Builder

The raw PyTorch graph is converted into a custom Intermediate Representation (IR). This abstraction decouples our compiler from PyTorch-specific constructs. The IR stores node types, extracted FP32 weights, biases, and structural metadata (kernel sizes, strides, padding) required for code generation.

```
============================================================
STEP 2: IR Construction
============================================================
IR Graph: SimpleCNN_MNIST
Total nodes: 12, Compute: 10
Parameters: 206,922, Weight mem: 808.3 KB

Name             | Op              | In Shape         | Out Shape        | W Shape
-----------------------------------------------------------------------------------------
x                | placeholder     | None             | (1, 1, 28, 28)   | -
conv1            | conv2d          | (1, 1, 28, 28)   | (1, 16, 28, 28)  | (16, 1, 3, 3)
relu1            | relu            | (1, 16, 28, 28)  | (1, 16, 28, 28)  | -
pool1            | maxpool2d       | (1, 16, 28, 28)  | (1, 16, 14, 14)  | -
conv2            | conv2d          | (1, 16, 14, 14)  | (1, 32, 14, 14)  | (32, 16, 3, 3)
relu2            | relu            | (1, 32, 14, 14)  | (1, 32, 14, 14)  | -
pool2            | maxpool2d       | (1, 32, 14, 14)  | (1, 32, 7, 7)    | -
flatten          | flatten         | (1, 32, 7, 7)    | (1, 1568)        | -
fc1              | linear          | (1, 1568)        | (1, 128)         | (128, 1568)
relu3            | relu            | (1, 128)         | (1, 128)         | -
fc2              | linear          | (1, 128)         | (1, 10)          | (10, 128)
output           | output          | (1, 10)          | (1, 10)          | -

Time  IR construction: 0.8 ms
```

Fig. 2. Step 2: IR Construction Output

C. Step 3: Graph Optimization

We apply graph-level transformations to reduce computational overhead:

- **Operator Fusion:** Sequences of convolutions or linear layers followed by ReLU activations are fused into single operations (`FusedConvReLU`, `FusedLinearReLU`). This eliminates the need to store intermediate tensors in memory and reduces loop overhead.

- **Dead Node Elimination:** Any nodes that do not contribute to the final output are identified via backward reachability analysis and pruned.

```
============================================================
STEP 3: Graph Optimization
============================================================
[Optimizer] Starting graph optimization...
[Optimizer]   Fused: conv1 + relu1 → fused_conv_relu
[Optimizer]   Fused: conv2 + relu2 → fused_conv_relu
[Optimizer]   Fused: fc1 + relu3 → fused_linear_relu
[Optimizer] Optimization complete: 10 → 7 compute nodes
[Optimizer] Stats: eliminated=0, fused=3, folded=0

Optimized IR:
IR Graph: SimpleCNN_MNIST
Total nodes: 12, Compute: 7
Parameters: 206,922, Weight mem: 808.3 KB

Name                | Op                  | In Shape             | Out Shape            | W Shape
--------------------------------------------------------------------------------------------------
x                   | placeholder         | None                 | (1, 1, 28, 28)       | -
conv1               | fused_conv_relu     | (1, 1, 28, 28)       | (1, 16, 28, 28)      | (16, 1, 3, 3)
pool1               | maxpool2d           | (1, 16, 28, 28)      | (1, 16, 14, 14)      | -
conv2               | fused_conv_relu     | (1, 16, 14, 14)      | (1, 32, 14, 14)      | (32, 16, 3, 3)
pool2               | maxpool2d           | (1, 32, 14, 14)      | (1, 32, 7, 7)        | -
flatten             | flatten             | (1, 32, 7, 7)        | (1, 1568)            | -
fc1                 | fused_linear_relu   | (1, 1568)            | (1, 128)             | (128, 1568)
fc2                 | linear              | (1, 128)             | (1, 10)              | (10, 128)
output              | output              | (1, 10)              | (1, 10)              | -

Time  Optimization: 0.0 ms
```

Fig. 3. Step 3: Graph Optimization Output

D. Step 4: INT8 Post-Training Quantization (PTQ)

To compress the model and accelerate inference, we convert all FP32 weights to 8-bit integers (INT8).

- **Symmetric Quantization:** We calculate a scale factor S for each weight tensor: $S = \max(|W|)/127$. Weights are then scaled and rounded: $W_{int8} = \text{round}(W_{float}/S)$.
- **Activation Quantization:** We simulate activation ranges (e.g., $[0, 6]$ for ReLU) to compute scaling factors and zero-points for intermediate activations, allowing the generated C code to use pure fixed-point arithmetic.

```
============================================================
STEP 4: INT8 Quantization
============================================================
[Quantizer] Starting INT8 quantization...
[Quantizer]   Quantized conv1: scale=0.002614, range=[-0.3314, 0.3320], mean_abs_error=0.000681
[Quantizer]   Quantized conv2: scale=0.000656, range=[-0.0833, 0.0833], mean_abs_error=0.000163
[Quantizer]   Quantized fc1: scale=0.000199, range=[-0.0253, 0.0253], mean_abs_error=0.000050
[Quantizer]   Quantized fc2: scale=0.000694, range=[-0.0882, 0.0879], mean_abs_error=0.000178

======================================================================
QUANTIZATION SUMMARY
======================================================================
Node                |  Orig (KB) | Quant (KB) |    Ratio
----------------------------------------------------------------------
conv1               |       0.62 |       0.20 |     3.1x
conv2               |      18.12 |       4.62 |     3.9x
fc1                 |     784.50 |     196.50 |     4.0x
fc2                 |       5.04 |       1.29 |     3.9x
----------------------------------------------------------------------
TOTAL               |     808.29 |     202.62 |     4.0x

Quantization complete! 808.3 KB → 202.6 KB

Time  Quantization: 2.5 ms
```

Fig. 4. Step 4: Quantization Output

E. Step 5: Memory Planning (Liveness Analysis)

To operate within the 256KB RAM constraint, we must optimize activation memory. The memory planner performs liveness analysis to determine when a tensor is no longer needed. It then employs a **buffer reuse** strategy, dynamically reallocating previously freed memory blocks to new tensors.

```
============================================================
STEP 5: Memory Planning
============================================================
============================================================
MEMORY PLANNING REPORT
============================================================
Weight Memory:             207,480 bytes (202.6 KB)
Peak Activation Memory:     12,544 bytes (12.2 KB)
Total Peak Memory:         220,024 bytes (214.9 KB)
Max RAM Limit:             262,144 bytes (256.0 KB)
Status:                 PASS

Buffer Reuse Stats:
  Without reuse: 43.0 KB
  With reuse:    12.2 KB
  Savings:       30.8 KB
  Buffers allocated: 1
  Buffers reused:    6

Per-Layer Breakdown:
Layer                |  Weights (B) | Activation (B) |   Cumulative
------------------------------------------------------------------------
conv1                |          208 |         12,544 |         12,544
pool1                |            0 |         12,544 |         25,088
conv2                |        4,736 |          6,272 |         31,360
pool2                |            0 |          6,272 |         37,632
flatten              |            0 |          6,272 |         43,904
fc1                  |      201,216 |            128 |         44,032
fc2                  |        1,320 |             10 |         44,042

Time  Memory planning: 0.1 ms
```

Fig. 5. Step 5: Memory Planning Output

F. Step 6: Hardware-Aware Latency Estimation

We model the computational cost of each layer based on a cycle-accurate estimation. For example, a Conv2D layer's cycle count is modeled as: $Cycles = OutputChannels \times InputChannels \times KernelSize^2 \times OutputHeight \times OutputWidth$ Using an assumed MCU clock frequency (e.g., 80 MHz), we estimate total inference time.

```
============================================================
STEP 6: Latency Estimation
============================================================
======================================================================
LATENCY ESTIMATION REPORT
======================================================================
MCU Clock: 80 MHz
Total Cycles:        1,255,809
Total Time:           15,697.6 µs (15.70 ms)

Layer                | Op Type              |       Cycles |  Time (µs) |     %
-------------------------------------------------------------------------------
conv1                | fused_conv_relu      |      125,440 |     1568.0 |  10.0%
pool1                | maxpool2d            |       12,544 |      156.8 |   1.0%
conv2                | fused_conv_relu      |      909,440 |    11368.0 |  72.4%
pool2                | maxpool2d            |        6,272 |       78.4 |   0.5%
flatten              | flatten              |            1 |        0.0 |   0.0%
fc1                  | fused_linear_relu    |      200,832 |     2510.4 |  16.0%
fc2                  | linear               |        1,280 |       16.0 |   0.1%

Time  Latency estimation: 0.0 ms

============================================================
STEP 7: C Code Generation
============================================================
[CodeGen]   Generated: generated/model_weights.h
[CodeGen]   Generated: generated/model_ops.h
[CodeGen]   Generated: generated/model.h
[CodeGen]   Generated: generated/model.c
[CodeGen] Generated C code in generated/

Time  Code generation: 34.0 ms
```

Fig. 6. Step 6: Latency Estimation Output

G. Step 7: C Code Generation (Backend)

Finally, the compiler generates pure, dependency-free C code. Weights are emitted as `static const int8_t` arrays stored in flash memory. Layer execution is unrolled into static function calls operating on pre-allocated static arrays (`buffer_a` and `buffer_b`), completely avoiding `malloc()` and ensuring deterministic execution.

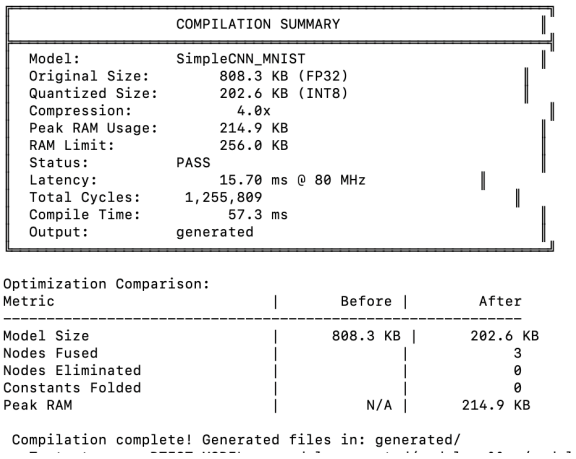

Fig. 7. Step 7 & Compilation Summary

III. Calculations and Performance Results

We validated the compiler using a Convolutional Neural Network (`SimpleCNN_MNIST`) designed for digit classification. The results demonstrate the efficacy of our optimizations.

A. 3.1 Memory Compression (Quantization)

- **Original FP32 Model Size:** 808.3 KB
- **Quantized INT8 Model Size:** 202.6 KB
- **Compression Ratio:** $808.3/202.6 \approx 4.0\times$
- *Result:* The model weights now comfortably fit within standard MCU flash limits.

B. 3.2 SRAM Optimization (Buffer Reuse)

- **Total Activation Memory (Without Reuse):** 43.0 KB
- **Peak Activation Memory (With Reuse):** 12.2 KB
- **Memory Savings:** $43.0 - 12.2 = 30.8$ KB (71.6% reduction)
- *Result:* The peak SRAM required during execution is just 214.9 KB (202.6 KB Weights + 12.2 KB Activations), successfully passing the strict 256.0 KB RAM limit.

C. 3.3 Latency Estimation

Assuming an ARM Cortex-M target running at 80 MHz (where 1 MAC operation $\approx$ 1 cycle):

- **Total Cycle Count:** 1,255,809 cycles
- **Estimated Inference Time:** $1,255,809 \text{ cycles}/80,000,000 \text{ cycles/sec} \approx 15.70$ ms
- *Result:* The model is capable of running real-time inference at over 60 frames per second on the edge device.

IV. Compilation Output & Verification

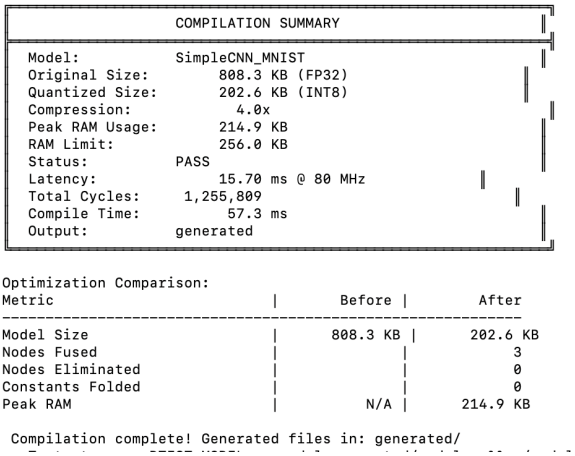

Fig. 8. Compilation Summary

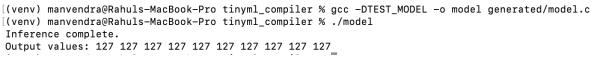

Fig. 9. Output Checker (gcc run)

V. Conclusion

The developed compiler successfully demonstrates that standard PyTorch models can be systematically transformed and heavily optimized for extreme edge environments. Through graph fusion, INT8 quantization, and intelligent memory planning, we achieved a 4x reduction in storage and a 71% reduction in dynamic memory, yielding a robust, deterministic C implementation ready for embedded deployment.